

Q1]

class A EXTENDS class B

•



inheritance / FullTimeEmployee EXTENDS Employee

•



implementations

class A implements class B

notification manager implements notification service

•



composition

class A depends on class B

•

class A uses class B but doesn't depend on

----->

Q2]

1) if we want to delete an hospital we have to delete all the departments because it's
↳ composition relationship

but if we want to delete a department, we should not delete all doctors because it's half composition

implements
[•] EmailNotifier ———> notifier

[•] order ~~has~~ ——— items

[•] order ———> customer / user

[•] orderService ~~has~~ ——— notifier

[•] orderService ———> user Invoice

1) Singleton → class should have only one reason to change

↳ Responsible of doing one mission

↳ but this class send email to boss but it's supposed to be responsible for report stuff

2) square as a child can't be interchangable with

The rectangle as a parent violates

Liskov principle

3) violates open for Extension closed for modification
↳ because if we add another customer type
↳ we will add new if statement

4) interface segregation violation
↳ class should only implement interfaces that will implement and use all of its methods

5) dependency inversion violation

user service depends on mysql
high level module low level module

both should depend on abstraction